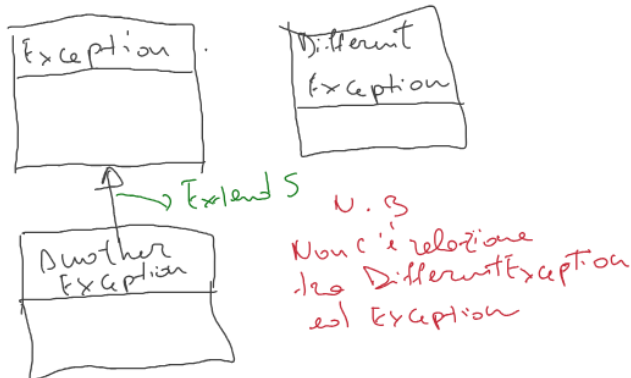


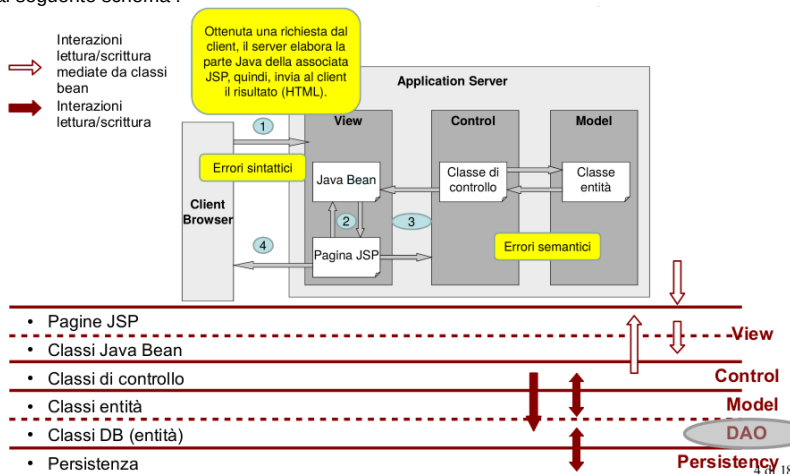
Lezione 13 18/12/23 Persistenza + state machine

venerdì 12 gennaio 2024 13:42

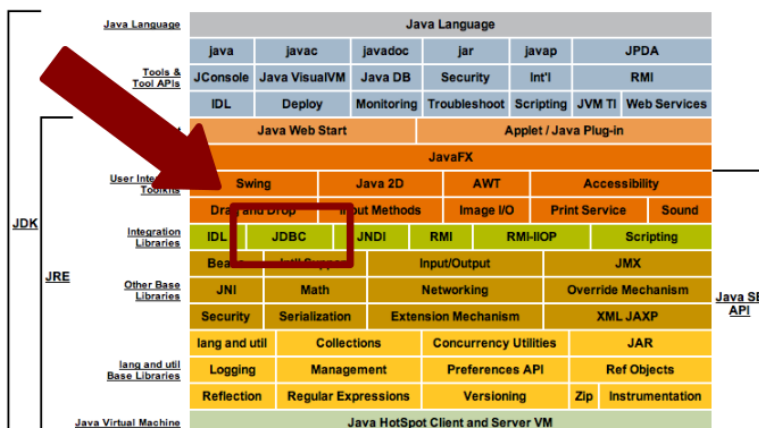
Prima di addentrarci nella persistenza, facciamo un breve riepilogo sulle eccezioni (condizioni di errore che alterano il flusso di azioni in logica di controllo), andando a vedere un esempio:



Ricordiamo inoltre che il **chiamante deve sapere come gestire l'errore**, levando la responsabilità al metodo di gestire la logica di errore. Vediamo un esempio: se usiamo i canali di error ed output, il programmatore deve fare attenzione all'ordinamento. Andiamo ora a vedere in dettaglio la **persistenza**: prima di addentrarci facciamo un flashback: il ciclo canonico di un oggetto comincia con invocazione del costruttore (allocazione in memoria dell'oggetto) e finisce con la de-allocazione (o terminazione di applicazione che ha costruito oggetto, distruzione da parte del GC). Più in generale si potrebbe avere bisogno di mantenere lo stato dell'oggetto oppure di migrare lo stesso oggetto tra più nodi oppure ripristinare lo stato dell'oggetto: **serializzazione / de-serializzazione**. Contestualizziamolo: per persistenza si intende il salvataggio di dati in una base di dati o in un file, che applicato al pattern mvc s'arriva al seguente schema:



Ricordiamo che le entità non possono interagire direttamente con il DAO, ma vi possono interagire attraverso i controller passando un parametro di tipo entità. Quindi per salvataggio di dati su file si deve usare opportuna convenzione di memorizzazione dei dati e del formato dei dati, mentre per il salvataggio dati su database si deve fare attenzione in quanto i dbms gestiscono raccolte di dati notevoli ed inoltre vi è mismatch tra rappresentazione dell'oggetto in o.o e oggetto in db: principalmente si usa questa rappresentazione. Andiamo ora a vedere come è possibile l'interazione con le basi di dati: si usa il JDBC (java data base connection), ovvero un framework per la connessione e interrogazione delle basi di dati, quindi usando questa tecnologia si ha agnosticismo tra applicazione e tipo di base di dati: quindi si parla di indipendenza da sw (in java) e persistenza. Questa libreria è inclusa nella jvm:



Andiamo a vedere ora in dettaglio il **JDBC** : si parla di un'api (application program interface) che consente di avere diverse tecnologie / implementazioni per potere interagire con le basi di dati (Postgres, mariadb, jdbc driver(connector j)). Questa api è formata da 4 macro classi:

1. JDBC-ODBC Bridge:
 - a. Open db connectivity : legacy standard per interconnettività di dbms
 - b. Invocazioni in jdbc sono mappate in odbc
2. Driver nativi :
 - a. Driver scritti in C, la loro invocazione richiede installazione locale : **dipendenza** applicazione da contesto (piattaforma di esecuzione)
3. Network protocol driver:
 - a. Scritti in java ma con possibilità di essere installati remotamente all'applicazione
 - b. Il driver ha la responsabilità/compiuto di fare da middleware verso uno o più dbms
4. Altri driver :
 - a. Driver scritti in java che risiedono sulla stessa macchina di dove viene fatta girare l'applicazione (driver risiedono nella jvm)
 - b. Usiamo questi

La struttura tipica di questo driver è la seguente :

1. load a driver for a specific DBMS
2. connect to a DBMS
3. foreach query to the DB on DBMS {
4. instantiate an interaction **Statement**
5. query the DB
6. do something with the results
7. }
8. close the connection with the DBMS

Buona pratica : mantenere il minimo numero di connessioni aperte con il db. In dettaglio :

1. Driver manager :
 - a. Creator per interazione con mysql

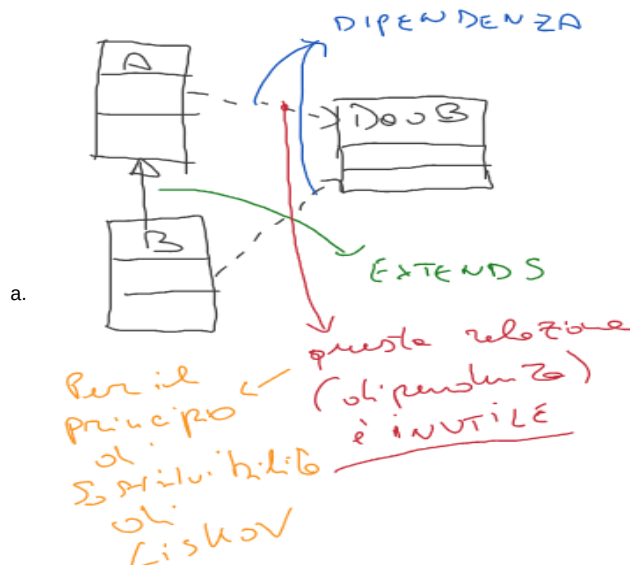

```
DriverManager.getConnection(
  b. "jdbc:mysql://address/database_name",
     "database_username", "database_password");
```
2. Statement :
 - a. Astrazione del concetto di operazione
 - b. Deve essere usato per incapsulare i dati provenienti dal db
 - c. Ogni statement deve essere istanziato su un'istanza della connessione col db
 - d. Quindi associa l'esecuzione della query con gli effettivi dati
 - e. `connection.createStatement()`
3. Result set:
 - a. Modellazione dei dati in forma tabellare dal db
 - b. Offre operazione **next()** che permette di iterare su tutti gli elementi della tabella
 - c. Possibile accedere ai singoli campi della tabella o per indice o per nome della colonna


```
String s = resultSetItem.getString(1); // preleva la
prima colonna come stringa
```
 - d.

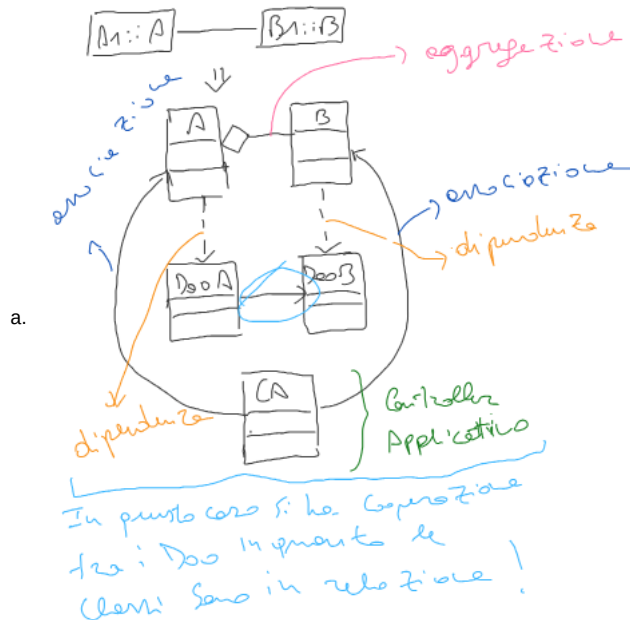

```
int eta = resultSetItem.getInt("eta"); // preleva la
colonna "età" come intero
```

Facciamo ora un paragone entità vs DAO : le prime rappresentano le astrazioni chiavi del dominio , estrapolate dall'analisi del dominio del problema e sono relazionate tra loro attraverso riferimenti in memoria , mentre le seconde sono **classi particolari** che mediano la rappresentazione di oggetto (o.o) nello strato di persistenza . Quindi il DAO ha la responsabilità di CREATOR di istanze di entità e ne configura i valori in memoria centrale secondo i valori delle entità. **Di solito ad un'entità corrisponde un DAO , con possibile comunicazione tra vari DAO.** Oltre a questo di solito (buona pratica) si cerca di separare la logica controllo da quella applicativa : implementazione delle query in altra classe. Vediamo degli esempi :

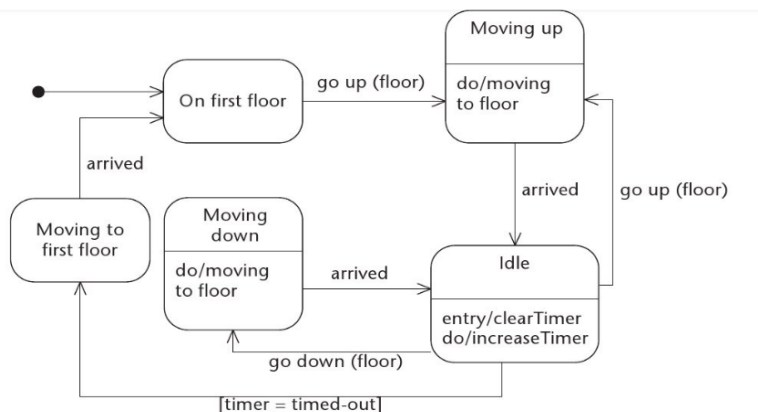
1. Relazione superflua



2. Serializzazione : usiamo i puntatori



Vediamo ora ultimo argomento : **gli state machine** : ricordiamo che questi diagrammi fanno parte dei diagrammi comportamentali (behavioural) . Ricordiamo che sono basati sulle macchine a stati finiti di Mealy e Moore : ovvero rappresentano la struttura dinamica di un sistema , o meglio come si comporta un oggetto reattivo . Riassumendo quindi , con questo diagramma è possibile rappresentare il comportamento degli oggetti e del sistema/sottosistema . In generale un diagramma di questo tipo è composto da archi e nodi (è una specie di grafo) , dove un nodo è caratterizzato da stato attività ed azione , mentre l'arco è caratterizzato da transizione, eventi , invocazioni di operazioni . **In generale un diagramma di questo tipo è caratterizzato da una 5-upla** . Per quanto riguarda invece lo stato si intende la situazione nella quale si trova in un determinato momento un oggetto e durante questo stato possono essere o meno soddisfatte condizioni , in base a valori attributi, istanze di relazioni con altri oggetti e/o attività che sta svolgendo. Vediamo prima un esempio e poi andiamo a vedere in dettaglio :

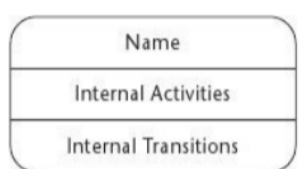


In dettaglio :

1. Stato :

- Rappresentazione dell'oggetto in un determinato istante
- Caratterizzato da nome , attività interne ed attività esterne

c.



- Dove il nome rappresenta una stringa di testo, attività interne sono le attività nel momento in cui si entra in quello stato (entry), transizioni interne sono le attività fatte nel momento in cui si è in quello stato (do) ed exit sono le attività/transizioni fatte nel momento in cui si esce.

2. Attività interne :

sono caratterizzare dai seguenti concetti:

event-name args-list '/' action-expression

- **entry**: attività eseguita nel momento in cui si entra in uno stato. Concettualmente è una attività atomica (i.e. non può essere interrotta)
- **exit**: attività eseguita nel momento in cui si esce da uno stato. Concettualmente è una attività atomica

a.

- **do**: identifica un'attività in esecuzione mentre l'oggetto è nello stato della transizione. La sua esecuzione può essere interrotta alla ricezione di un evento

- (**include**: invocazione a una submachine. l'azione contiene il nome della submachine invocata)

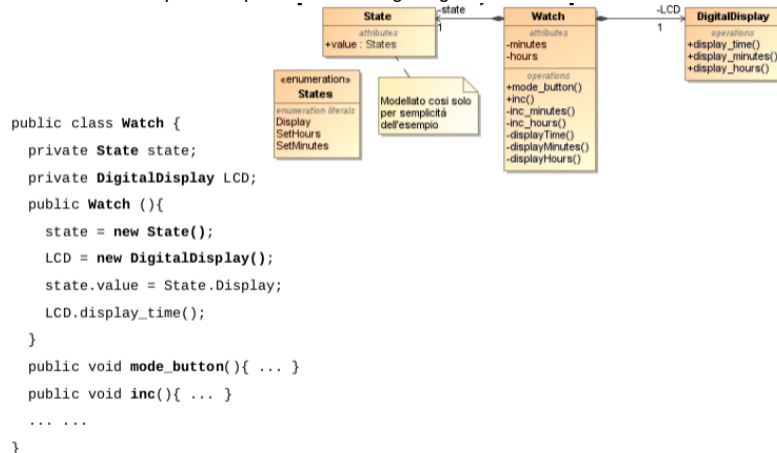
3. Transizioni :

- È una relazione tra due o più stati
- Indica quindi un cambiamento di stato da parte di un oggetto in seguito al verificarsi o meno di una condizione
- Esce dallo stato di partenza ed entra in quella di arrivo
- Queste etichette servono ad esportare le operazioni della classe.**
- Composta da : 5-upla
 - Stato sorgente
 - Evento scatenante
 - Condizione di guardia
 - Effetto : azione o generatori di eventi
 - Stato destinatario

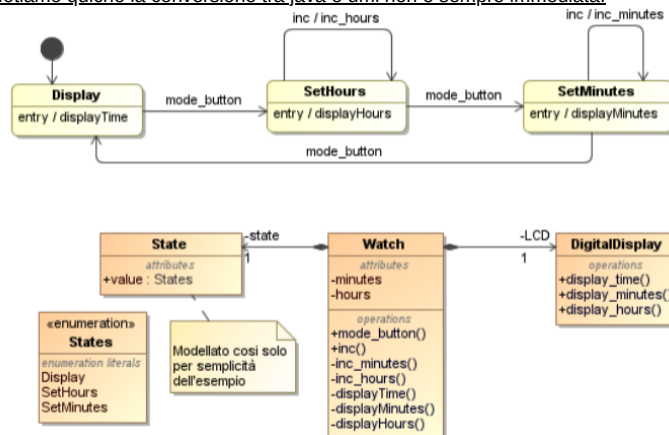
4. Comportamenti :

- Può essere specificato tramite algoritmo , state machine oppure entrambi i modi
- Da notare che il passaggio da state machine a o.o. è difficile : in quanto non vi è la rappresentazione/gestione degli eventi, piuttosto si ha una rappresentazione agnostica , ed infine il modello uml è ambiguo.

Vediamo un esempio : comportamento orologio digitale



Notiamo quiche la conversione tra java e uml non è sempre immediata.



Ricordiamo che gli eventi dello state vengono esportati come operazioni pubbliche. Una domanda potrebbe sorgere : se ricevo un evento come faccio a sapere in quale stato sono? Difficile dare una risposta, e quindi per questo devo implementare ogni evento pubblico con uno switch :

```

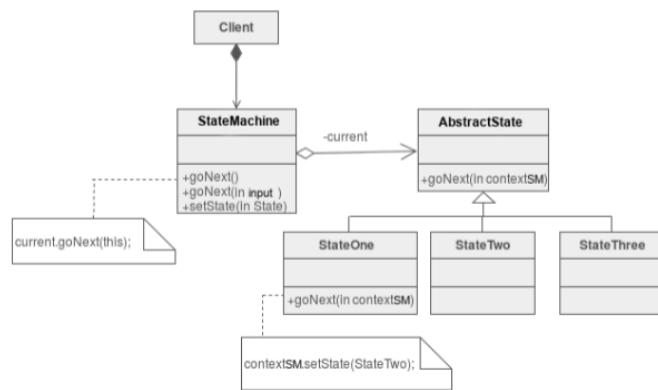
public void mode_button(){
    switch (state.value){
        case State.Display:
            //no cond., no exit, no effects
            state.value=State.SetHours;
            this.displayHours();
            break;
        case State.SetHours:
            //no cond., no exit, no effects
            state.value=State.SetMinutes;
            this.displayMinutes();
            break;
        case State.SetMinutes:
            //no cond., no exit, no effects
            state.value = State.Display;
            this.displayTime();
            break;
    }
}

public void inc() {
    switch (state.value){
        case State.Display :
            //nothing to do
            break;
        case State.SetHours:
            //no cond., no exit,
            this.inc_hours();
            state.value = State.SetHours;
            this.displayHours();
            break;
        case State.SetMinutes:
            //no cond., no exit,
            this.inc_minutes();
            state.value = State.SetMinutes;
            this.displayMinutes();
            break;
    }
}

```



Come già accennato, questa rappresentazione del sistema, la possiamo vedere come pattern : il pattern gof state:



In dettaglio :

1. Scopo : consentire ad un oggetto di cambiare il suo comportamento a seguito di un cambiamento interno
2. Chiamato anche strategy pattern
3. Per quanto riguarda il "context" (abstract state) -> state machine rappresenta il contratto tra client ed il comportamento che si vuole modellare
4. Specifica gli stati in modo astratto , i quali saranno implementati nelle sotto classi
5. Lo stato corrente viene specificato come stato astratto nella classe astratta
6. Per quanto riguarda la gestione degli stati , vengono gestiti in tutte le classe (vedasi switch)
7. Se aggiungo/rimuovo nuovo stato basta aggiungere/rimuovere sottoclasse
8. Possibile inoltre che la state machine abbia metodi inaccessibili al client per funzionamento proprio.
9. Vediamo ora quindi lo state finale:

